

Рекомендации IEEE по разработке требований к программному обеспечению (IEEE Std 830-1998 (Ревизия IEEE Std 830-1993))

Спонсор: Комитет по стандартам в области программной инженерии Компьютерного сообщества IEEE.

Одобрено: 25 июня 1998 года Советом по стандартам IEEE-SA.

Реферат: Описаны содержание и характеристики качественной спецификации требований к программному обеспечению (software requirements specification, SRS), приведены несколько примеров плана SRS. Эти рекомендации ориентированы на составление спецификаций требований к разрабатываемому программному обеспечению, но также могут помочь при выборе коммерческих и разработанных самостоятельно программных продуктов. Также приведены указания по согласованию со стандартом IEEE/EIA 12207.1-1997.

Введение

(Данное введение не является составной частью Стандарта)

Данные рекомендации описывают рекомендованный подход к разработке спецификаций требований к программному обеспечению. Они основываются на модели, в которой результатом процесса разработки спецификации требований к программному обеспечению является непротиворечивый и полный документ. Они призваны помочь:

1. Потребителям программного обеспечения — точно описать, что они желают получить;
2. Поставщикам программного обеспечения — в точности понять, что хочет потребитель;
3. Специалистам — выполнить следующие задачи:
 1. Разработать план стандартной спецификации требований к программному обеспечению (SRS) для нужд вашей конкретной организации;
 2. Определить формат и содержание своих собственных спецификаций требований к программному обеспечению; Разработать собственные дополнительные инструменты поддержки, такие, как чек-листы качества SRS или справочники разработчиков SRS.

Для потребителей, поставщиков и прочих специалистов качественная SRS должна обеспечить несколько преимуществ, в частности:

- Установить основу для соглашения между потребителями и поставщиками о том, что должен делать программный продукт. Полное описание выполняемых программным обеспечением

функций, заданное в SRS, поможет потенциальным пользователям определить, удовлетворяет ли программное обеспечение их потребности, либо каким образом его следует модифицировать для удовлетворения их потребностей.

- Снизить трудоемкость разработки. Подготовка SRS заставляет различные заинтересованные группы в организации-потребителе тщательно рассмотреть все требования до начала разработки и уменьшить последующие перепроектирование, перекодирование и перетестирование. Внимательный пересмотр требований в SRS может выявить упущения, недоразумения и несогласованности на ранних стадиях цикла разработки, когда эти проблемы легче исправить.
- Обеспечить основу для оценки стоимости и сроков. Описание разрабатываемого продукта, приведенное в SRS, является реалистичной основой для оценки затрат на проектирование и может быть использовано для обоснования цены.
- Обеспечить основу для валидации и верификации. Руководствуясь хорошей SRS, организации могут гораздо продуктивнее разработать собственные планы валидации и верификации. Как часть контракта на разработку SRS обеспечивает эталон, соответствие которому может быть оценено.
- Облегчить передачу. SRS облегчает передачу программного обеспечения новым пользователям или на новые машины. Таким образом, потребители обнаруживают, что упрощается передача программного обеспечения в другие подразделения их организации, а поставщики обнаруживают, что облегчается его передача новым потребителям.
- Служить основой для усовершенствования. Поскольку в SRS обсуждается сам продукт, а не проект его разработки, SRS служит основой для последующего усовершенствования завершенного продукта. Может потребоваться изменение SRS, но она все равно является основой для дальнейшей оценки продукта.

Читатели данного документа могут обратиться к Приложению Б за указаниями по использованию данных рекомендаций в соответствии с требованиями стандарта IEEE/EIA 12207.1-1997.

(Опущен весьма объемистый список лиц, которым документ обязан своим появлением на свет).

1. Обзор

Данные рекомендации описывают рекомендованный подход к составлению спецификаций требований к программному обеспечению. Они делятся на 5 разделов.

1. Раздел 1 поясняет область применимости рекомендаций.
2. Раздел 2 перечисляет ссылки на другие стандарты.
3. В разделе 3 приведены определения основных используемых терминов.

4. Раздел 4 предоставляет дополнительную информацию по написанию хорошей SRS.
5. В разделе 5 обсуждаются все основные части SRS.

Данные рекомендации содержат также два приложения; в одном представлены альтернативные форматы шаблонов, в другом — указания по соответствию со стандартом IEEE/EIA 12207.1-1997.

1.1. Область применимости

Данный документ представляет собой рекомендации по написанию спецификаций требований к программному обеспечению. Он описывает содержание и характеристики качественной спецификации требований к программному обеспечению (SRS) и представляет несколько примеров плана SRS.

Данные рекомендации ориентированы на специфицирование требований к разрабатываемому программному обеспечению, но могут также оказаться полезны при выборе самостоятельно разработанных и коммерческих продуктов. Однако их применение к уже разработанному программному обеспечению может отрицательно повлиять на продуктивность.

Если программное обеспечение встроено в некоторую большую систему, например, медицинское оборудование, то могут возникнуть проблемы, лежащие вне пределов применимости данных рекомендаций.

Данные рекомендации описывают процесс создания продукта и содержание продукта. Продуктом является SRS. Рекомендации могут быть непосредственно использованы для создания SRS или как модель для более специфического стандарта.

Данная рекомендация не предлагает никаких конкретных методов или инструментов для подготовки SRS.

2. Ссылки

(Опущен перечень ссылок на другие стандарты).

3. Определения

В общем определения терминов, используемых в данных рекомендациях, соответствуют определениям, приведенным в IEEE Std 610.12-1990. Ниже приведены определения ключевых терминов, используемых в данных рекомендациях.

Контракт: Юридический документ, согласованный между потребителем и поставщиком. Он включает технические и организационные требования, цены и сроки поставки продукта. Контракт может также включать неформальную, но полезную информацию, например, обязательства или ожидания заинтересованных сторон.

Потребитель: Персона (персоны), которая оплачивает продукт и обычно (но не обязательно) определяет требования к нему. В контексте этих рекомендаций потребитель и поставщик могут принадлежать к одной и той же организации.

Поставщик: Персона (персоны), которая производит продукт для потребителя. В контексте этих рекомендаций потребитель и поставщик могут принадлежать к одной и той же организации.

Пользователь: Персона (персоны), которая непосредственно управляет или взаимодействует с продуктом. Пользователь (-ли) и потребитель (-ли) зачастую не являются одной и той же персоной.

4. Рекомендации по производству качественных SRS

Этот раздел представляет информацию, которую следует иметь в виду при написании SRS. Она включает следующее:

1. Природа SRS;
2. Окружение SRS;
3. Характеристики качественной SRS;
4. Совместная подготовка SRS;
5. Эволюция SRS;
6. Прототипирование;
7. Встраивание разработки в SRS;
8. Встраивание проектных требований в SRS.

4.1. Природа SRS

SRS — это спецификация отдельного программного продукта, программы или набора программ, которые выполняют некоторые действия в некоторой среде. SRS может быть написана одним или несколькими представителями поставщика, одним или несколькими представителями потребителя, или совместно представителями обеих сторон. Подраздел 4.4 рекомендует участие в этом процессе обеих сторон.

Основные вопросы, которые должны решить авторы SRS, следующие:

1. Функциональность. Что должно по замыслу делать программное обеспечение?
2. Внешние интерфейсы. Как программное обеспечение взаимодействует с людьми, системным оборудованием, другим оборудованием и другими программами?
3. Производительность. Каковы скорость, доступность, время ответа, время восстановления различных программных функций и т.д.?
4. Атрибуты. Каковы переносимость, корректность, пригодность к поддержке, безопасность и т.д.?
5. Проектные ограничения, налагаемые на реализацию. Есть ли требования к действующим стандартам, языкам реализации, политикам поддержания целостности баз данных, ограничениям на ресурсы, операционной среде и т.д.?

Авторам SRS следует избегать включения требований к проектированию и разработке в SRS.

Рекомендуемое содержание SRS приведено в разделе 5.

4.2. Окружение SRS

Важно сознавать роль, которую SRS играет в общем плане проекта и которая определена в IEEE Std 610.12-1990. Программное обеспечение может содержать полностью всю функциональность проекта либо быть частью более крупной системы. В последнем случае обычно бывает SRS, устанавливающая интерфейсы между системой и ее программной частью и определяющая внешние требования к производительности и функциональности программной части. Разумеется, впоследствии SRS должна согласовываться с этими системными требованиями и детализироваться на их основе.

IEEE STD 1074-1997 описывает этапы жизненного цикла программного обеспечения и допустимые входные данные для каждого этапа. Другие стандарты, например, перечисленные в разделе 2, ссылаются на другие части жизненного цикла программного обеспечения и таким образом могут дополнять требования к программному обеспечению.

Поскольку SRS играет особую роль в процессе разработки программного обеспечения, авторы SRS должны проявлять осторожность и не выходить за границы этой роли. Это значит, что SRS

1. должна корректно определять все требования к программному обеспечению. Требование к программному обеспечению может существовать благодаря природе решаемой задачи либо благодаря специфической особенности проекта.
2. не должна описывать никаких деталей разработки и реализации. Их следует описывать на стадии разработки проекта.
3. не должна накладывать дополнительные ограничения на программное обеспечение. Они должны задаваться в других документах, например, плане обеспечения качества программного обеспечения.

Таким образом, правильно написанная SRS ограничивает диапазон допустимых проектных решений, но не задает никакого конкретного решения.

4.3. Характеристики качественной SRS

SRS должна быть:

1. корректной;
2. непротиворечивой;
3. полной;
4. целостной;
5. упорядоченной по важности и/или стабильности;
6. верифицируемой;
7. модифицируемой;
8. трассируемой.

4.3.1. Корректность

SRS является корректной в том и только том случае, если программное обеспечение действительно должно удовлетворять каждому установленному требованию.

Не существует инструментов или процедур, которые обеспечивали бы корректность. Следует сравнивать SRS со всеми подходящими вышестоящими спецификациями, например, спецификациями требований к системе, с другой программной документацией и другими подходящими стандартами, с целью обеспечить их согласованность. Кроме того, потребитель или пользователь могут определить, корректно ли SRS отражает их действительные потребности. Трассируемость делает эту процедуру более простой и менее подверженной ошибкам (см. п.п. 4.3.8).

4.3.2. Непротиворечивость

SRS является непротиворечивой в том и только том случае, если каждое установленное требование имеет только одну интерпретацию. Как минимум это означает, что каждая характеристика конечного продукта должна описываться одним и тем же уникальным термином. В случаях, когда термин, используемый в данном контексте, может иметь несколько значений, следует включить термин в глоссарий, где его смысл уточняется.

SRS — важная часть процесса управления требованиями в жизненном цикле программного обеспечения, и она используется при разработке, реализации, отслеживании проекта, верификации проекта, а также при обучении, как описано в IEEE STD 1074-1977. SRS должна быть непротиворечива как с точки зрения того, кто ее создает, так и с точки зрения того, кто ее использует. Однако эти группы зачастую имеют различные точки зрения и поэтому не стремятся описывать требования к программному обеспечению одинаковым образом. Представление, которое улучшает спецификацию требований с точки зрения разработчика, может быть непродуктивным, если оно ухудшает понимание со стороны пользователя, и наоборот.

П.п. с 4.3.2.1 по 4.3.2.3 рекомендуют, как избегать неоднозначности.

4.3.2.1. Ловушка естественного языка

Требования часто пишутся на естественном языке (например, английском). Естественные языки неоднозначны по своей природе. SRS на естественном языке должна быть пересмотрена независимой стороной с целью выявления неоднозначного использования языка, которое должно быть исправлено.

4.3.2.2. Языки спецификации требований

Один из способов избежать неоднозначности, свойственной естественным языкам, — это написать SRS на особом языке спецификации требований. Его языковые процессоры автоматически находят многие лексические, синтаксические и семантические ошибки.

Одним из недостатков при использовании подобных языков является продолжительность времени, необходимого для их изучения. Кроме того, многие пользователи без технической подготовки находят их непонятными. Напротив, эти языки более выразительны для некоторых типов требований и некоторых типов систем. Таким образом, они могут оказывать тонкое влияние на требования.

4.3.2.3 Инструментарий представления

В целом методы разработки требований, языки и инструменты их поддержки делятся на три основные категории — объект, процесс и поведение. Объектно-ориентированные подходы организуют требования в терминах объектов реального мира, их атрибутов и сервисов, выполняемых этими объектами. Процесс-ориентированные подходы организуют требования в иерархии функций, которые взаимодействуют посредством потоков данных. Поведенческие подходы описывают внешнее поведение системы в терминах некоторой абстрактной нотации (такой, как исчисление предикатов), математических функций или конечных автоматов.

Степень, в которой подобные инструменты и методы могут быть полезны при подготовке SRS, зависят от размера и сложности программы. В данном документе не предпринимаются попытки описать или рекомендовать какой-либо конкретный инструмент.

При использовании любого из этих подходов лучше всего придерживаться описания на естественном языке. В этом случае потребители, незнакомые с нотациями, все же будут способны понимать SRS.

4.3.3. Полнота

SRS является полной в том и только том случае, если она включает в себя следующие элементы:

1. Все значимые требования, имеющие отношение к функциональности, производительности, проектным ограничениям, атрибутам или внешним интерфейсам. В частности, следует принимать во внимание все внешние требования, налагаемые системной спецификацией.
2. Определение всех ответов программного обеспечения на все возможные классы входных данных во всех возможных классах ситуаций. Отметим, что важно задавать ответы как на допустимые, так и на недопустимые входные значения.
3. Все метки и ссылки на все рисунки, таблицы и диаграммы в SRS, а также определения всех терминов и единиц измерения.

4.3.3.1. Использование TBD

Любая SRS, в которой используется фраза «следует определить» (“to be determined”, TBD), не является полной. Впрочем, иногда TBD необходимы и должны сопровождаться:

1. описанием условий, вызвавших TBD (например, почему ответ неизвестен), чтобы ситуация могла быть разрешена;

2. описанием того, что должно быть сделано для удаления TBD, кто ответственный за это удаление и к какому сроку оно должно быть удалено.

4.3.4. Целостность

Понятие «целостность» относится к внутренней целостности. Если SRS не согласуется с каким-либо из документов высшего уровня, например, спецификацией требований к системе, она не является корректной (см. п.п. 4.3.1).

4.3.4.1. Внутренняя целостность

SRS является внутренне целостной в том и только том случае, если никакое подмножество описанных в ней отдельных требований не конфликтуют друг с другом. Ниже приведены три наиболее вероятных типа конфликтов в SRS:

1. Специфические характеристики объектов реального мира могут конфликтовать. Например:
 1. Формат вывода отчета может быть описан в одном требовании как таблица, а в другом — как текст.
 2. Одно требование может устанавливать, что все лампочки должны быть зелеными, а другое — синими.
2. Может быть логический или временной конфликт между двумя заданными действиями. Например:
 1. Одно требование может задавать, что программа должна складывать два введенных числа, а другое — что она должна перемножать их.
 2. Одно требование может устанавливать, что «А» должно всегда следовать за «Б», в то время как другое требует, чтобы «А» и «Б» происходили одновременно.
3. Два или более требований могут описывать одни и те же объекты реального мира, но использовать при этом разные термины. Например, запрос программой пользовательского ввода может называться «подсказкой» в одном требовании и «приглашением» в другом. Использование стандартной терминологии и определений способствует повышению целостности.

4.3.5. Упорядоченность по важности и/или стабильности

SRS является упорядоченной по важности и/или стабильности, если каждое требование в ней имеет идентификатор, указывающий либо важность, либо стабильность данного требования. Как правило, не все требования, относящиеся к программному продукту, являются одинаково важными. Одни требования могут быть принципиально важными, особенно для жизненно-важных приложений, тогда как другие могут быть просто желательными.

Каждое требование в SRS должно быть идентифицировано с целью сделать эти различия отчетливыми и явными. Идентификация требований таким образом помогает:

1. потребителям — более тщательно рассмотреть каждое требование, что зачастую проясняет их неявные предположения.
2. разработчикам — принимать правильные проектные решения и соответственно распределять усилия, затрачиваемые на различные части программного продукта.

4.3.5.1. Степень стабильности

Один из методов идентификации требований использует оценку стабильности. Стабильность может быть выражена как число ожидаемых изменений требования, оцениваемое на основе опыта или знания предстоящих событий, которые повлияют на организацию, функции или персонал, использующий программную систему.

4.3.5.2. Степень необходимости

Другой способ ранжировать требования — разделить классы требований на обязательные, желательные и необязательные.

1. Обязательные. Подразумевается, что программное обеспечение не будет приемлемо, если требование не выполняется надлежащим образом.
2. Желательные. Подразумевается, что данные требования улучшат программный продукт, но их отсутствие не делает программный продукт неприемлемым.
3. Необязательные. Подразумевается класс функций, которые могут оказаться полезными, а могут и не оказаться. Это дает поставщику возможность предложить что-то, выходящее за пределы SRS.

4.3.6. Верифицируемость

SRS является верифицируемой в том и только том случае, если каждое требование является верифицируемым. Требование является верифицируемым в том и только том случае, если существует некий конечный достаточно эффективный процесс, согласно которому человек либо машина может проверить, удовлетворяет ли программный продукт данному требованию. В общем случае никакое неоднозначное требование не является верифицируемым.

Неверифицируемые требования включают такие обороты, как «работает хорошо», «хороший человеко-машинный интерфейс» или «обычно должно происходить». Эти требования невозможно верифицировать, поскольку невозможно точно установить смысл терминов «хорошо», «хороши» или «обычно». Утверждение «программа никогда не должна входить в бесконечный цикл» не является верифицируемой, поскольку его тестирование невозможно даже теоретически.

Пример верифицируемого утверждения:

Вывод программы должен производиться в пределах 20 секунд в 60% случаев, и должен производиться в пределах 30 секунд в 100% случаев.

Данное утверждение может быть верифицировано, поскольку в нем используются конкретные термины и измеряемые величины.

Если невозможно придумать способ определения, удовлетворяет ли программное обеспечение данному требованию, это требование должно быть удалено или пересмотрено.

4.3.7. Модифицируемость

SRS является модифицируемой в том и только том случае, если ее структура и стиль таковы, что любые изменения требования могут быть сделаны легко, полно и целостно без изменений этих структуры и стиля. Модифицируемость обычно требует от SRS:

1. иметь связную и легкую в использовании организацию с оглавлением, индексом и явными перекрестными ссылками;
2. не быть избыточным (например, одно и то же требование не должно появляться более чем в одном месте в SRS);
3. представлять каждое требование отдельно, не смешивая с другими требованиями.

Сама по себе избыточность ошибкой не является, но легко может привести к ошибке. Порой избыточность помогает сделать SRS более читаемой, однако может возникнуть проблема при обновлении избыточного документа. Например, требование может быть изменено лишь в одном месте среди тех, где оно появляется. В этом случае SRS перестает быть целостной. Если избыточность действительно необходима, следует включать в SRS явные перекрестные ссылки, чтобы сделать ее модифицируемой.

4.3.8. Трассируемость

SRS является трассируемой, если источник каждого требования ясен и если она в дальнейшем позволит ссылаться на каждое требование в документации разработки. Рекомендуются следующие два типа трассируемости:

1. Обратная трассируемость (т.е. к предыдущим этапам разработки). Она зависит от наличия в каждом требовании явной ссылки на его источник в предшествующей документации.
2. Прямая трассируемость (т.е. ко всем документам, порожденным SRS). Она зависит от наличия в SRS уникального имени или ссылочного номера для каждого требования.

Прямая трассируемость SRS особенно важна, когда программный продукт переходит в фазу использования и поддержки. Поскольку код и проектная документация модифицируемы, важно иметь возможность уточнить весь набор требований, которые могут быть затронуты этими изменениями.

4.4. Совместная подготовка SRS

Процесс разработки программного обеспечения должен начинаться соглашением между поставщиком и потребителем о том, что именно должно делать готовое программное обеспечение. Это соглашение, оформленное в виде SRS, должно быть подготовлено совместно. Это важно, поскольку обычно ни потребитель, ни поставщик по отдельности не имеют достаточной квалификации для самостоятельного написания хорошей SRS.

1. Потребители обычно не имеют представления о разработке программного обеспечения в объеме, достаточном для написания полезной SRS.
2. Поставщики обычно не имеют понимания задач потребителя и предметной области в степени, достаточной для задания требований к системе удовлетворительного качества.

Таким образом, потребитель и поставщик должны работать вместе для создания хорошо написанной и совершенно понятной SRS.

Особая ситуация возникает, когда одновременно определяется система и ее программное обеспечение. В этом случае функциональность, интерфейсы, производительность и прочие атрибуты и ограничения программного обеспечения не предопределены, а скорее определяются совместно и являются предметом соглашений и изменений. Это делает достижение характеристик, перечисленных в подразделе 4.3, более трудным, но не менее важным. В частности, SRS, которая не согласуется с требованиями родительской системы, является некорректной.

В данных рекомендациях не обсуждаются особый стиль, использование языка или технология хорошего написания. Однако весьма важно, чтобы SRS была написана хорошо. В качестве руководства можно использовать общие руководства по написанию технических текстов.

4.5. Эволюция SRS

SRS может потребовать развития по мере продвижения процесса разработки программного продукта. Может оказаться невозможным задать некоторые детали во время начала проекта (например, может быть невозможно определить все экранные формы для интерактивных программ в фазе сбора требований). Дополнительные изменения могут возникнуть по мере обнаружения недостатков, дефектов и неточностей в SRS.

При этом необходимо учитывать два важных аспекта:

1. Требования должны быть заданы настолько полно и тщательно, насколько возможно в данный момент, даже если очевидно, что впоследствии неизбежен их пересмотр. Следует отметить факт их неполноты.
2. Следует начать формальный процесс для идентификации, управления, отслеживания и учета запланированных изменений. Одобренные изменения должны вноситься в SRS таким образом, чтобы:
 1. обеспечивать точное и полное протоколирование изменений;
 2. позволять пересмотр текущих и замененных частей SRS.

4.6. Прототипирование

Прототипирование часто используется на проектном этапе работы с требованиями. Существует множество инструментов для быстрого и легкого изготовления прототипов, демонстрирующих некоторые характеристики системы. См. также ASTM E1340-96.

Прототипы полезны по следующим причинам:

1. Потребитель может предпочесть посмотреть и оценить прототип, нежели читать и оценивать SRS. Поэтому прототип обеспечивает быструю обратную связь.
2. Прототип демонстрирует неожиданные аспекты поведения системы. Таким образом, он не только дает ответы, но и задает новые вопросы. Это помогает более полно проанализировать SRS.
3. SRS, основанная на прототипе, имеет тенденцию меньше подвергаться изменениям во время разработки, тем самым сокращая время разработки.

Прототип следует использовать как способ извлечения требований.

Некоторые характеристики, наподобие форм отчетов, могут быть непосредственно извлечены из прототипа. Другие требования могут возникнуть в ходе экспериментирования с прототипом.

4.7. Встраивание разработки в SRS

Требование задает видимую извне функцию или атрибут системы. Разработка описывает некоторую составную часть системы и/или ее интерфейсы с остальными частями. Авторы SRS должны отчетливо различать идентификацию требуемых проектных ограничений и разработку проекта. Заметим, что каждое требование в SRS ограничивает выбор проектных решений. Тем не менее, из этого не следует, что каждое требование есть разработка.

SRS должна задавать, какие функции необходимо выполнить над какими данными, какой получить результат, где и для кого. SRS должна фокусироваться на выполняемых сервисах. Обычно SRS не должна задавать конкретных проектных решений, например:

1. разбиение программного обеспечения на модули;
2. назначение функций модулям;
3. описание потоков информации или управления между модулями;
4. выбор структур данных.

4.7.1. Необходимые требования к разработке

В особых случаях некоторые требования могут существенно ограничить разработку. Например, требования к секретности или безопасности могут непосредственно отразиться на разработке в виде необходимости

1. хранить некоторые функции в отдельных модулях;
2. позволять только ограниченные коммуникации между некоторыми областями программы;
3. проверять целостность данных для критических переменных.

Примерами допустимых ограничений разработки являются физические требования, требования к производительности, стандарты на разработку программного обеспечения и стандарты по обеспечению качества.

Таким образом, требования следует устанавливать с чисто внешней точки зрения. Используя модели для иллюстрации требований, помните, что модель отражает лишь внешнее поведение и не задает никаких конкретных проектных решений.

4.8. Встраивание проектных требований в SRS

SRS должна быть направлена на сам программный продукт, а не на процесс его изготовления. Проектные требования представляют соглашения между потребителем и поставщиком по поводу деталей контракта, относящихся к производству программного обеспечения, поэтому их не следует включать в SRS. Обычно в их число входят:

1. Стоимость;
2. График поставки;
3. Процедуры отчетности;
4. Методики разработки программного обеспечения;
5. Обеспечение качества;
6. Критерии валидации и верификации;
7. Процедуры приемки.

Проектные требования задаются в других документах, обычно в плане разработки программного обеспечения, плане обеспечения качества или в общих положениях проекта.

5. Части SRS

В этом разделе обсуждается каждая основная часть SRS. Эти части упорядочены ниже в виде плана, который может использоваться как образец при написании SRS. Хотя SRS не обязана в точности следовать этому плану или использовать такие же заголовки частей, хорошая SRS должна включать в себя всю приведенную информацию.

Содержание SRS:

1. Введение
 1. Назначение
 2. Область применения
 3. Определения, акронимы и сокращения
 4. Обзор
2. Общее описание
 1. Позиционирование продукта
 2. Функции продукта
 3. Пользовательские характеристики
 4. Ограничения
 5. Предположения и зависимости

3. Специфические требования (см. п.п. с 5.3.1 по 5.3.8 с пояснением возможных специфических требований. См. также в Приложении А различные способы организации этого раздела SRS)
4. Приложения
5. Индекс

5.1. Введение (Раздел 1 SRS)

Введение должно представлять обзор SRS В целом. Оно должно включать следующие подразделы:

1. Назначение.
2. Область применения.
3. Определения, акронимы и сокращения.
4. Ссылки.
5. Обзор.

5.1.1. Назначение (1.1 SRS)

В этом подразделе следует:

1. Определить назначение SRS;
2. Задать целевую аудиторию SRS.

5.1.2. Область применения (1.2 SRS)

Этот подраздел должен:

1. Идентифицировать производимый продукт по имени (например, Host DBMS, Report Generator и т.д.);
2. Пояснять, что должен делать программный продукт, а также, при необходимости, чего он не должен делать;
3. Описать применение программного обеспечения, включая выгоды, намерения и цели;
4. Согласовываться со сходными положениями спецификаций верхнего уровня (например, спецификацией требований к системе), если они существуют.

5.1.3. Определения, акронимы и сокращения (1.3 SRS)

Этот подраздел должен представлять определения всех терминов, акронимов и сокращений, необходимых для правильной интерпретации SRS. Эта информация может быть представлена в виде ссылок на одно или более приложений к SRS либо на другие документы.

5.1.4. Ссылки (1.4 SRS)

Данный подраздел должен:

1. Представлять полный перечень документов, на которые есть ссылки где-либо в SRS;
2. Идентифицировать каждый документ по названию, отчетному номеру (если применимо), дате и опубликовавшей организации;
3. Задавать источники, из которых могут быть получены документы, на которые имеются ссылки.

Эта информация может быть представлена в виде ссылки на приложение или другой документ.

5.1.5. Обзор (1.5 SRS)

Данный подраздел должен:

1. Описывать содержимое остальной части SRS;
2. Пояснять организацию SRS.

5.2. Общее описание (Раздел 2 SRS)

Этот раздел SRS должен описывать общие факторы, оказывающие влияние на продукт и требования к нему. В этом разделе не приводятся специфические требования. В нем подготавливается основа для требований, которые детально определяются в Разделе 3 SRS, и приводится информация, облегчающая их понимание.

Этот раздел обычно состоит из шести подразделов:

1. Позиционирование продукта;
2. Функции продукта;
3. Характеристики пользователей;
4. Ограничения;
5. Предположения и зависимости;
6. Распределение требований.

5.2.1. Позиционирование продукта (2.1 SRS)

Этот подраздел позиционирует продукт среди других связанных продуктов. Если продукт является независимым и полностью самодостаточным, это следует отразить здесь. Если SRS определяет продукт, который является компонентом большей системы, как это часто бывает, то данный подраздел должен связать требования к большей системе с функциональностью программного обеспечения и определить интерфейсы между системой и программным обеспечением.

Могут быть полезными блок-схемы, показывающие основные компоненты большей системы, связи между ними и внешние интерфейсы.

Этот подраздел должен также описывать, как программное обеспечение работает под действием различных ограничений. Например, эти ограничения могут включать:

1. Системные интерфейсы;
2. Пользовательские интерфейсы;
3. Аппаратные интерфейсы;
4. Программные интерфейсы;
5. Коммуникационные интерфейсы;
6. Память;
7. Операции;
8. Требования к адаптируемости на месте.

5.2.1.1. Системные интерфейсы

Следует перечислить все системные интерфейсы и идентифицировать функциональность программного обеспечения для выполнения требований к системе, а также описание интерфейса для соответствия системе.

5.2.1.2. Пользовательские интерфейсы

Следует задать:

1. Логические характеристики каждого интерфейса между программным продуктом и его пользователями. Сюда входят конфигурационные характеристики (например, требуемые экранные формы, страницы или раскладки окон, содержимое всех отчетов или меню, а также доступность программируемых функциональных клавиш), необходимые для выполнения программных требований.
2. Все аспекты оптимизации интерфейса с персоналом, который должен использовать систему. Они могут просто включать список, как система должна выглядеть с точки зрения пользователя, а как не должна. Например, может быть требование наличия опции полных или кратких сообщений об ошибках. Подобно всем остальным, эти требования должны быть верифицируемыми, например: «оператор 4-го разряда должен научиться выполнять действие X за Z минут после часового обучения», а не «оператор должен выполнять действие X». (Это также можно задать в «Системных атрибутах программного обеспечения» в разделе под названием «Простота использования»).

5.2.1.3. Аппаратные интерфейсы

Здесь следует задать логические характеристики каждого интерфейса между программным продуктом и аппаратными компонентами системы. Сюда входят конфигурационные характеристики (число портов, набор инструкций и т.д.). Также освещаются такие моменты, как: какие устройства поддерживаются, как они поддерживаются, а также протоколы. Например, поддержка терминала может задавать поддержку полноэкранного интерфейса, в противоположность построчному вводу.

5.2.1.4. Программные интерфейсы

Следует задать использование прочих необходимых программных продуктов (например, система управления данными, операционная система или математический пакет), а также интерфейсы с другими прикладными системами (например, связь между системой приема оплаты счетов и общей бухгалтерской системой). Для каждого необходимого программного продукта следует предоставить следующую информацию:

1. Название;
2. Мнемоника;
3. Номер спецификации;
4. Номер версии;
5. Источник.

Для каждого интерфейса следует предоставить следующую информацию:

1. Обсуждение назначения интерфейсного программного обеспечения с точки зрения программного продукта.
2. Определение интерфейса в терминах содержания и формата сообщений. Не обязательно детально описывать хорошо документированные интерфейсы, но требуется ссылка на документ, определяющий требуемый интерфейс.

5.2.1.5. Коммуникационные интерфейсы

Следует задать различные интерфейсы коммуникаций: протоколы локальных сетей и т.д.

5.2.1.6. Ограничения по памяти

Следует задать все значимые характеристики и ограничения, касающиеся первичной и вторичной памяти.

5.2.1.7. Операции

Следует задать обычные и специфические операции, которые требуются пользователю, например:

1. Различные модели операций в организации пользователя (например, операции, инициируемые пользователем);
2. Периоды интерактивных операций и периоды операций, не требующих ручного вмешательства;
3. Функции поддержки обработки данных;
4. Операции резервного копирования и восстановления.

Примечание. Иногда они задаются как часть раздела «Пользовательские Интерфейсы».

5.2.1.8. Требования к адаптации на месте

Следует:

1. Определить требования ко всем данным или последовательностям инициализации, специфичным для данного места, задачи или режима работы (например, таблицы значений, безопасные пределы и т.д.);
2. Задать особенности, относящиеся к месту или задаче, которые следует модифицировать с целью адаптации программного обеспечения к конкретной инсталляции.

5.2.2. Функции продукта (2.2 SRS)

В этом подразделе SRS следует представить сводку основных функций, выполняемых системой. Например, SRS для бухгалтерской программы может посвятить эту часть работе со счетами, обслуживанию клиентов и подготовке платежных поручений, не вдаваясь в обширную детализацию этих функций.

Иногда сводка функций, необходимых для данной части, берется прямо из соответствующего раздела спецификации верхнего уровня (если она есть), которая размещает некоторые функции в программном продукте. Заметим для ясности, что:

1. Функции должны быть организованы таким образом, чтобы сделать перечень функций понятным потребителю или другим читателям при первом прочтении документа.
2. Можно использовать текстовые или графические методики для представления различных функций и отношений между ними. Подобные диаграммы не должны представлять реализацию продукта, а лишь показывать логические взаимосвязи между переменными.

5.2.3. Характеристики пользователей (2.3 SRS)

Этот подраздел SRS должен описывать общие характеристики предполагаемых пользователей, включая уровень образования, опыт и техническую грамотность. В нем не следует устанавливать специфические требования, но следует привести причины, по которым некоторые специфические требования заданы далее в Разделе 3 SRS.

5.2.4. Ограничения (2.4 SRS)

В этом подразделе должны быть приведены общие описания всего того, что может ограничить действия разработчика. Они включают:

1. Правовые вопросы;
2. Аппаратные ограничения (например, требования к длительности сигналов);
3. Интерфейсы с другими приложениями;
4. Параллельные операции;
5. Функции аудита;
6. Функции управления;
7. Языковые ограничения высшего порядка;
8. Протоколы синхронизации сигналов (например, XON-XOFF, ACK-NACK);
9. Требования к надежности;
10. Критичность приложения;
11. Соображения безопасности и секретности.

5.2.5. Предположения и зависимости (2.5 SRS)

В этом подразделе следует перечислить все факторы, которые влияют на требования, устанавливаемые SRS. Эти факторы не являются проектными ограничениями, а скорее относятся к их изменениям, которые могут повлиять на требования SRS. Например, может быть сделано предположение, что некоторая операционная система будет доступна для оборудования, на которое ориентирован программный продукт. Если в действительности операционная система недоступна, потребуется соответствующее изменение SRS.

5.2.6. Распределение требований (2.6 SRS)

Этот подраздел должен представлять требования, которые могут быть отложены до будущих версий системы.

5.3. Специфические требования (Раздел 3 SRS)

Этот раздел SRS должен содержать все требования к программному обеспечению на уровне детализации, достаточном, чтобы позволить разработчикам создать систему, удовлетворяющую этим ограничениям, а тестерам — проверить, удовлетворяет ли система этим ограничениям. В данном разделе каждое требование должно быть ориентировано на пользователей, операторов или другие системы, внешние по отношению к данной. Эти требования должны включать минимальное описание для каждого ввода в систему, каждого ответа системы, а также всех функций, выполняемых системой в ответ на ввод или для поддержки вывода. Поскольку зачастую эта часть SRS является самой большой и наиболее важной, применяются следующие принципы:

1. Специфические требования должны соответствовать характеристикам, описанным в подразделе 4.3.
2. Специфические требования должны снабжаться перекрестными ссылками на ранние документы.
3. Все требования должны быть уникально идентифицируемыми.
4. Следует уделить особое внимание организации требований с целью достижения максимальной читабельности.

Перед изучением способов организации требований полезно рассмотреть различные элементы, включающие в себя требования, которые описаны в п.п. с 5.3.1 по 5.3.7.

5.3.1. Внешние интерфейсы

Здесь должно быть детальное описание всех входных и выходных данных программной системы. Оно должно дополнять описания интерфейсов из подраздела 5.2 и не содержать повторяющейся информации. Оно должно иметь формат и содержание, как указано ниже:

1. Имя элемента;
2. Описание назначения;
3. Источник входных данных или получатель выходных;
4. Допустимый диапазон, точность и отклонения;
5. Единицы измерения;
6. Временные диаграммы;
7. Взаимосвязи с другими входами/выходами;
8. Форматы и организация экранов;
9. Форматы и организация окон;
10. Форматы данных;
11. Форматы команд;
12. Завершающие сообщения.

5.3.2. Функции

Функциональные требования должны определять фундаментальные действия, которые должны выполняться программным обеспечением для приема и обработки входных данных, а также обработки и вывода

выходных данных. Они обычно перечисляются в виде предложений, начинающихся со слов «Система должна...».

Они включают:

1. Проверку допустимости входных значений;
2. Точный порядок действий;
3. Реакцию на нештатные ситуации, включающие:
 1. Переполнение;
 2. Коммуникационные проблемы;
 3. Обработку ошибок и восстановление;
4. Влияние параметров;
5. Взаимосвязь между входными и выходными данными, включая:
 1. Порядок ввода/вывода;
 2. Формулы преобразования входных данных в выходные

Может оказаться удобным разделение функциональных требований на подфункции или подпроцессы. Это не значит, что программный проект будет разделен таким же образом.

5.3.3. Требования к производительности

Этот подраздел должен задавать как статические, так и динамические численные требования, предъявляемые в целом к программному обеспечению или к взаимодействию человека с программой. Статические численные требования могут включать следующие:

1. Число поддерживаемых терминалов;
2. Число одновременно поддерживаемых пользователей;
3. Объем и тип обрабатываемой информации.

Статические численные требования иногда оформляются в виде отдельного раздела.

Динамические численные требования могут включать, например, число транзакций и задач, или объем данных, обрабатываемых в некоторый период данных в условиях как нормальной, так и пиковой нагрузки.

Все эти требования следует формулировать в терминах измеримых величин.

Например:

95% транзакций должны обрабатываться менее чем за 1 секунду
вместо:

Оператор не должен ждать, пока завершится транзакция.

Примечание. Численные границы, применимые к отдельной функции, обычно задаются в описании данной функции, в подразделе обработки.

5.3.4. Логические требования к базе данных

Здесь следует задать логические требования к информации, которая должна размещаться в базе данных. Они могут включать следующие:

1. Типы информации, используемой различными функциями;
2. Частоту использования;

3. Возможность доступа;
4. Сущности и отношения между ними;
5. Ограничения целостности;
6. Требования к хранению данных.

5.3.5. Ограничения проектирования

Здесь задаются ограничения проектирования, налагаемые другими стандартами, аппаратурой и т.д.

5.3.5.1. Соответствие стандартам

Этот подраздел должен задавать ограничения, вытекающие из существующих стандартов и правил. Они могут включать следующее:

1. Форматы отчетов;
2. Именованье данных;
3. Бухгалтерские процедуры;
4. Протоколирование работы.

Например, требования к программному обеспечению по трассировке вычислительных действий. Подобные трассировки необходимы для некоторых приложений для выполнения требований правовых или финансовых стандартов. Требование протоколирования работы может, например, устанавливать, что при изменениях в платежной ведомости прежнее и новое значения должны записываться в файл трассировки.

5.3.6. Атрибуты программной системы

Некоторые атрибуты программного обеспечения могут служить требованиями. Важно, чтобы требуемые атрибуты были заданы таким образом, чтобы можно было объективно проверить выполнение данного требования. В п.п. с 5.3.6.1 по 5.3.6.5 приведен список примеров.

5.3.6.1. Надежность

Следует перечислить факторы, необходимые для установления требуемого уровня надежности программного обеспечения.

5.3.6.2. Доступность

Следует перечислить факторы, призванные гарантировать определенный уровень доступности системы, такие как контрольные точки, восстановление и перезапуск.

5.3.6.3. Безопасность

Следует перечислить факторы, защищающие программное обеспечение от случайного или злонамеренного доступа, использования, модификации, разрушения или разглашения. Специфические требования в этой области включают необходимость:

1. Использования криптографии;
2. Хранение логов или истории;
3. Назначать некоторые функции различным модулям;

4. Ограничивать коммуникации между некоторыми областями программы;
5. Проверять целостность данных для критических переменных.

5.3.6.4. Поддерживаемость

Следует перечислить атрибуты программного обеспечения, относящиеся к легкости поддержки самого программного обеспечения. Это могут быть некоторые требования, относящиеся к модульности, интерфейсам, сложности и т.д. Не следует помещать здесь требования лишь потому, что принято считать их хорошим тоном разработки.

5.3.6.5. Переносимость

Следует перечислить атрибуты программного обеспечения, касающиеся легкости переноса программного обеспечения на другие компьютеры и/или операционные системы. Они могут включать следующее:

1. Доля компонентов с машинно-зависимым кодом;
2. Доля машинно-зависимого кода;
3. Использование испытанного переносимого языка;
4. Использование особенного компилятора или подмножества языка;
5. Использование особенной операционной системы.

5.3.7. Организация специфических требований

Для любой нетривиальной системы детальные требования обширно разрастаются. Поэтому рекомендуется тщательный подход к их организации в виде, оптимальном для понимания. Не существует единой организации, оптимальной для всех систем. Различные классы систем придерживаются различной организации требований в Разделе 3 SRS. Некоторые способы организации описаны в п.п. с 5.3.7.1 по 5.3.7.7.

5.3.7.1. Режим системы

Поведение некоторых систем кардинально различается в зависимости от режима системы. Например, система управления может иметь различные наборы функций для разных режимов: обучение, нормальный или аварийный. При организации этого раздела по режимам, можно использовать планы А.1 или А.2. Выбор определяется тем, зависят ли от режима интерфейсы и производительность.

5.3.7.2. Класс пользователя

Некоторые системы обеспечивают различные наборы функций для разных классов пользователей. Например, система управления лифтом предоставляет различные возможности пассажирам, обслуживающему персоналу и пожарным. При организации этого раздела по классам пользователей следует использовать план А.3.

5.3.7.3. Объекты

Объекты — это сущности реального мира, которые имеют соответствие в системе. Например, в системе мониторинга пациентов в число объектов входят пациенты, датчики, медсестры, кабинеты, врачи, медикаменты и т.д. С каждым объектом связаны наборы атрибутов (данного объекта) и функций (выполняемых данным объектом). Эти функции называются также сервисами, методами или процессами. При организации этого раздела по объектам следует использовать план А.4. Заметим, что наборы объектов могут иметь общие атрибуты и сервисы. Они группируются вместе как классы.

5.3.7.4. Функциональные возможности

Функциональная возможность — это внешний сервис системы, который может потребовать последовательность ввода данных для достижения желаемого результата. Например, в телефонной системе функциональные возможности включают местный вызов, переадресацию звонка и конференцию. Обычно функциональные возможности описывают в виде последовательности пар запрос-ответ. При организации этого раздела в соответствии с функциональными возможностями следует использовать план А.5.

5.3.7.5. Внешние воздействия

Некоторые системы могут быть лучше организованы путем описания их функций в терминах внешних воздействий. Например, функции системы автоматического приземления самолета могут быть разбиты на секции для потери тяги, порывов ветра, неожиданной смены курса, избыточной вертикальной скорости и т.д. При организации этого раздела в соответствии с внешними воздействиями следует использовать план А.6.

5.3.7.6. Отклики

Некоторые системы могут быть лучше организованы путем описания всех функций, поддерживающих генерацию откликов. Например, функции системы управления персоналом могут быть разбиты на секции, соответствующие всем функциям, связанным с генерацией чеков оплаты, всем функциям, связанным с генерацией текущего списка сотрудников, и т.д. Следует использовать план А.6, в котором все внешние воздействия заменены откликами.

5.3.7.7. Функциональная иерархия

Если ни одна из вышеперечисленных организационных схем не подходит, общая функциональность может быть организована в виде иерархии функций, организованных в соответствии с входными данными, выходными данными или внутренними данными. Могут быть использованы диаграммы потоков данных и словари данных, чтобы показать взаимосвязи между функциями и данными. При организации этого раздела в соответствии с функциональной иерархией следует использовать план А.7.

5.3.8. Дополнительные комментарии

В процессе работы над SRS могут оказаться применимыми более одной организационной схемы из перечисленных в п.п. 5.3.7.7. В таких случаях организуйте специфические требования по нескольким иерархиям, приспособленным под специфические нужды системы, для которой разрабатываются спецификации. Например, см. A.8 как пример организации, комбинирующей классы пользователей и функциональные возможности. Все дополнительные требования могут быть размещены в отдельном разделе в конце SRS.

Имеется множество нотаций, методик и автоматизированных инструментов для поддержки документирования требований. В основном они полезны в части организационной функции. Например, при организации по режимам могут оказаться полезными конечные автоматы диаграммы состояний; при организации по объектам может оказаться полезным объектно-ориентированный анализ; при организации по функциональным возможностям могут оказаться полезными последовательности запрос-ответ, а при организации по функциональной иерархии могут пригодиться диаграммы потоков данных и словари данных.

В любом из планов, описанных с A.1 по A.8, разделы с именем «Функциональное требование i» могут быть описаны на естественном языке, на псевдокоде, на языке описания систем либо в виде четырех подразделов с именами «Введение», «Входные данные», «Обработка» и «Выходные данные».

5.4. Вспомогательная информация

Вспомогательная информация делает SRS более удобочитаемой. Она включает следующее:

1. Оглавление;
2. Индекс;
3. Приложения.

5.4.1. Оглавление и индекс

Оглавление и индекс крайне важны и должны следовать общим принципам композиции.

5.4.2. Приложения

Приложения не всегда рассматриваются как часть SRS и не всегда необходимы. Они могут включать:

- Примеры форматов входных и выходных данных, описания примеров ценового анализа или результаты пользовательских обзоров;
- Вспомогательная и дополнительная информация, которая может помочь читателю SRS;
- Описание проблемы, решаемой программным обеспечением;

- Специальные инструкции по упаковке для кода и носителей для соответствия требованиям по безопасности, экспорту начальной загрузке и т.д.

Если приложения включены в состав SRS, следует явно указать, являются ли они частью требований.

Приложения

Приложение А. Образцы SRS

А.1. Образец Раздела 3 SRS, организованный по режимам: версия 1

3. Специфические требования

- 3.1. Внешние требования к интерфейсу
 - 3.1.1. Пользовательские интерфейсы
 - 3.1.2. Аппаратные интерфейсы
 - 3.1.3. Программные интерфейсы
 - 3.1.4. Коммуникационные интерфейсы
- 3.2. Функциональные требования
 - 3.2.1. Режим 1
 - 3.2.1.1. Функциональное требование 1.1
 - ...
 - 3.2.1.n. Функциональное требование 1.n
 - 3.2.2. Режим 2
 - ...
 - 3.2.m. Режим m
 - 3.2.m.1. Функциональное требование m.1
 - ...
 - 3.2.m.n. Функциональное требование m.n
- 3.3. Требования к производительности
- 3.4. Проектные ограничения
- 3.5. Атрибуты программной системы
- 3.6. Прочие требования

А.2. Образец Раздела 3 SRS, организованный по режимам: версия 2

3. Специфические требования

- 3.1. Функциональные требования
 - 3.1.1. Режим 1
 - 3.1.1.1. Внешние интерфейсы
 - 3.1.1.1.1. Пользовательские интерфейсы
 - 3.1.1.1.2. Аппаратные интерфейсы
 - 3.1.1.1.3. Программные интерфейсы
 - 3.1.1.1.4. Коммуникационные интерфейсы
 - 3.1.1.2. Функциональные требования
 - 3.1.1.2.1. Функциональное требование 1
 - ...
 - 3.1.1.2.n. Функциональное требование n
 - 3.1.2. Режим 2
 - ...
 - 3.1.m. Режим m
- 3.2. Проектные ограничения
- 3.3. Атрибуты программной системы
- 3.4. Прочие требования

А.3. Образец Раздела 3 SRS, организованный по классам пользователей

3. Специфические требования

- 3.1. Внешние требования к интерфейсу
 - 3.1.1. Пользовательские интерфейсы
 - 3.1.2. Аппаратные интерфейсы
 - 3.1.3. Программные интерфейсы
 - 3.1.4. Коммуникационные интерфейсы
- 3.2. Функциональные требования
 - 3.2.1. Класс пользователей 1
 - 3.2.1.1. Функциональное требование 1.1
 - ...
 - 3.2.1.n. Функциональное требование 1.n
 - 3.2.2. Класс пользователей 2
 - ...
 - 3.2.m. Класс пользователей m
 - 3.2.m.1. Функциональное требование m.1
 - ...
 - 3.2.m.n. Функциональное требование m.n
- 3.3. Требования к производительности
- 3.4. Проектные ограничения
- 3.5. Атрибуты программной системы
- 3.6. Прочие требования

А.4. Образец Раздела 3 SRS, организованный по объектам

3. Специфические требования

- 3.1. Внешние требования к интерфейсу
 - 3.1.1. Пользовательские интерфейсы
 - 3.1.2. Аппаратные интерфейсы
 - 3.1.3. Программные интерфейсы
 - 3.1.4. Коммуникационные интерфейсы
- 3.2. Классы/объекты
 - 3.2.1. Класс/объект 1
 - 3.2.1.1. Атрибуты (непосредственные или унаследованные)
 - 3.2.1.1.1. Атрибут 1
 - ...
 - 3.2.1.1.n. Атрибут n
 - 3.2.1.2. Функции (сервисы и методы, непосредственные или унаследованные)
 - 3.2.1.2.1. Функциональное требование 1.1
 - ...
 - 3.2.1.2.m. Функциональное требование 1.m
 - 3.2.1.3. Сообщения (принимаемые или отправляемые)
 - 3.2.2. Класс/объект 2
 - ...
 - 3.2.p. Класс/объект p
- 3.3. Требования к производительности
- 3.4. Проектные ограничения
- 3.5. Атрибуты программной системы
- 3.6. Прочие требования

A.5. Образец Раздела 3 SRS, организованный по Функциональным возможностям

3. Специфические требования

- 3.1. Внешние требования к интерфейсу
 - 3.1.1. Пользовательские интерфейсы
 - 3.1.2. Аппаратные интерфейсы
 - 3.1.3. Программные интерфейсы
 - 3.1.4. Коммуникационные интерфейсы
- 3.2. Функциональные возможности системы
 - 3.2.1. Системная функция 1
 - 3.2.1.1. Введение (назначение функции)
 - 3.2.1.2. Последовательность «запрос/ответ»
 - 3.2.1.3. Связанные функциональные требования
 - 3.2.1.3.1. Функциональное требование 1
 - ...
 - 3.2.1.3.n. Функциональное требование n
 - 3.2.2. Системная функция 2
 - ...
 - 3.2.m. Системная функция m
- 3.3. Требования к производительности
- 3.4. Проектные ограничения
- 3.5. Атрибуты программной системы
- 3.6. Прочие требования

А.6. Образец Раздела 3 SRS, организованный по внешним воздействиям

3. Специфические требования

3.1. Внешние требования к интерфейсу

- 3.1.1. Пользовательские интерфейсы
- 3.1.2. Аппаратные интерфейсы
- 3.1.3. Программные интерфейсы
- 3.1.4. Коммуникационные интерфейсы
- 3.2. Функциональные требования
 - 3.2.1. Воздействие 1
 - 3.2.1.1. Функциональное требование 1.1
 - ...
 - 3.2.1.n. Функциональное требование 1.n
 - 3.2.2. Воздействие 2
 - ...
 - 3.2.m. Воздействие m
 - 3.2.m.1. Функциональное требование m.1
 - ...
 - 3.2.m.n. Функциональное требование m.n
- 3.3. Требования к производительности
- 3.4. Проектные ограничения
- 3.5. Атрибуты программной системы
- 3.6. Прочие требования

А.7. Образец Раздела 3 SRS, организованный по функциональной иерархии

3. Специфические требования

- 3.1. Внешние требования к интерфейсу
 - 3.1.1. Пользовательские интерфейсы
 - 3.1.2. Аппаратные интерфейсы
 - 3.1.3. Программные интерфейсы
 - 3.1.4. Коммуникационные интерфейсы
- 3.2. Функциональные требования
 - 3.2.1. Информационные потоки
 - 3.2.1.1. Диаграмма потоков данных 1
 - 3.2.1.1.1. Сущности
 - 3.2.1.1.2. Процессы
 - 3.2.1.1.3. Топология
 - 3.2.1.2. Диаграмма потоков данных 2
 - 3.2.1.2.1. Сущности
 - 3.2.1.2.2. Процессы
 - 3.2.1.2.3. Топология
 - ...
 - ...
 - 3.2.1.n. Диаграмма потоков данных n
 - 3.2.1.n.1. Сущности
 - 3.2.1.n.2. Процессы
 - 3.2.1.n.3. Топология
 - 3.2.2. Описание процессов
 - 3.2.2.1. Процесс 1
 - 3.2.2.1.1. Входные данные
 - 3.2.2.1.2. Алгоритм или формула обработки
 - 3.2.1.1.3. Результирующие данные
 - 3.2.2.2. Процесс 2
 - 3.2.2.2.1. Входные данные
 - 3.2.2.2.2. Алгоритм или формула обработки
 - 3.2.2.2.3. Результирующие данные
 - ...
 - 3.2.2.m. Процесс m
 - 3.2.2.m.1. Входные данные
 - 3.2.2.m.2. Алгоритм или формула обработки
 - 3.2.2.m.3. Результирующие данные
 - 3.2.3. Спецификации структур данных
 - 3.2.3.1. Структура 1
 - 3.2.3.1.1. Тип записи
 - 3.2.1.1.2. Поля записи
 - 3.2.3.2. Структура 2
 - 3.2.3.2.1. Тип записи
 - 3.2.1.2.2. Поля записи
 - ...

- 3.2.3.р. Структура р
 - 3.2.3.р.1. Тип записи
 - 3.2.1.р.2. Поля записи
- 3.2.4. Словарь данных
 - 3.2.4.1. Элемент данных 1
 - 3.2.4.1.1. Имя
 - 3.2.4.1.2. Представление
 - 3.2.4.1.3. Единицы/Форматы
 - 3.2.4.1.4. Точность
 - 3.2.4.1.5. Диапазон
 - 3.2.4.2. Элемент данных 2
 - 3.2.4.2.1. Имя
 - 3.2.4.2.2. Представление
 - 3.2.4.2.3. Единицы/Форматы
 - 3.2.4.2.4. Точность
 - 3.2.4.2.5. Диапазон
 - ...
 - 3.2.4.q. Элемент данных q
 - 3.2.4.q.1. Имя
 - 3.2.4.q.2. Представление
 - 3.2.4.q.3. Единицы/Форматы
 - 3.2.4.q.4. Точность
 - 3.2.4.q.5. Диапазон
- 3.3. Требования к производительности
- 3.4. Проектные ограничения
- 3.5. Атрибуты программной системы
- 3.6. Прочие требования

А.8. Образец Раздела 3 SRS, демонстрирующий множественную организацию

3. Специфические требования

- 3.1. Внешние требования к интерфейсу
 - 3.1.1. Пользовательские интерфейсы
 - 3.1.2. Аппаратные интерфейсы
 - 3.1.3. Программные интерфейсы
 - 3.1.4. Коммуникационные интерфейсы
- 3.2. Функциональные требования
 - 3.2.1. Класс пользователей 1
 - 3.2.1.1. Функция 1.1
 - 3.2.1.1.1. Введение (назначение функции)
 - 3.2.1.1.2. Последовательность «запрос/ответ»
 - 3.2.1.1.3. Связанные функциональные требования.
 - 3.2.1.2. Функция 1.2
 - 3.2.1.2.1. Введение (назначение функции)
 - 3.2.1.2.2. Последовательность «запрос/ответ»
 - 3.2.1.2.3. Связанные функциональные требования.
 - ...
 - 3.2.1.m. Функция 1.m
 - 3.2.1.m.1. Введение (назначение функции)
 - 3.2.1.m.2. Последовательность «запрос/ответ»
 - 3.2.1.m.3. Связанные функциональные требования.
 - 3.2.2. Класс пользователей 2
 - ...
 - 3.2.n. Класс пользователей n
- 3.3. Требования к производительности
- 3.4. Проектные ограничения
- 3.5. Атрибуты программной системы
- 3.6. Прочие требования

Приложение В. Указания по соответствию требованиям IEEE/EIA 12207.1-1997

В.1 Обзор

Комитет стандартов в области программной инженерии (the Software Engineering Standards Committee, SESC) при IEEE Computer Society утвердил политику принятия международных стандартов. В 1995 году был составлен международный стандарт ISO/IEC 12207, «Информационные технологии — Процессы жизненного цикла». Стандарт устанавливает общие положения процессов жизненного цикла программного обеспечения с хорошо определенной терминологией, на которую может ссылаться индустрия программного обеспечения.

В 1995 году SESC разработал ISO/IEC 12207 и решил, что этот стандарт следует одобрить и использовать как основу процессов жизненного цикла в составе IEEE Software Engineering Collection. Адаптацией IEEE стандарта ISO/IEC 12207 является IEEE/EIA 12207.0-1996. Она содержит ISO/IEC 12207 со следующими добавлениями: улучшенный подход к совместимости, назначение процессов жизненного цикла, назначение данных жизненного цикла, список исправлений.

Реализация ISO/IEC в IEEE также включает следующее:

- IEEE/EIA 12207.1-1997, IEEE/EIA Руководство по информационной технологии. Процессы жизненного цикла программного обеспечения. Данные жизненного цикла.
- IEEE/EIA 12207.2-1997, IEEE/EIA Руководство по информационной технологии. Процессы жизненного цикла программного обеспечения. Замечания по реализации.
- Дополнения к 11 стандартам SESC (IEEE STDs 730, 828, 829, 830, 1012, 1016, 1058, 1062, 1219, 1233, 1362) с целью определения корреляции между документами, выполненными в соответствии с существующими стандартами SESC, и документами, выполненными в соответствии с IEEE/EIA 12207.1-1997.

Примечание. Хотя IEEE/EIA 12207.1-1997 является руководством, он также содержит положения, применимые как стандарт с определенными требованиями. Данное приложение трактует 12207.1-1997 как стандарт.

В.2. Корреляция

Данный раздел поясняет отношения между IEEE STD 830-1998, IEEE/EIA 12207.0-1996 и IEEE/EIA 12207.1-1997 в следующих областях: терминология, процесс и данные жизненного цикла.

В.2.1. Корреляция терминологии

Как данные рекомендации, так и IEEE/EIA 12207.0-1996 придают сходные значения таким ключевым терминам, как «программное обеспечение», «требования», «поставщик», «разработчик», «персонал сопровождения». Данные рекомендации используют термин «потребитель»,

когда IEEE/EIA 12207.0-1996 использует «заказчик», и «пользователь», когда IEEE/EIA 12207.0-1996 использует «оператор».

В.2.2. Корреляция процессов

IEEE/EIA 12207.0-1996 использует процесс-ориентированный подход к описанию набора требований к программному обеспечению. Данные рекомендации используют подход, ориентированный на продукт, при этом продуктом является «Описание требований к программному обеспечению» (Software Requirements Description, SRD). Это естественные этапы процесса, а именно этапы создания каждой части SRD. Этому можно найти соответствие в требованиях к процессу из IEEE/EIA 12207.0-1996. Различие состоит в том, что данные рекомендации сфокусированы на разработку требований к программному обеспечению, в то время как IEEE/EIA 12207.0-1996 представляет обзор всего жизненного цикла и упоминает анализ требований к программному обеспечению как часть процесса разработки. В данных рекомендациях произведена существенная детализация всего, что относится к подготовке SRD.

В.2.3. Корреляция данных жизненного цикла

IEEE/EIA 12207.0-1996 устанавливает точку зрения, что требования к программному обеспечению вытекают из требований к системе. Поэтому он использует термин «описание» вместе «спецификация» при описании требований к программному обеспечению. В случае системы, в которой программное обеспечение является одним из компонентов, каждый из которых требует спецификации, следовало бы разработать спецификацию требований к системе (System Requirements Specification, SRS) и одну или более SRD. При использовании термина «спецификация требований к программному обеспечению» (Software Requirements Specification) возникло бы недоразумение, что означает SRS — требования к системе или к программному обеспечению. В случае, если программная система является автономной, IEEE/EIA 12207.1-1997 гласит: «Если программное обеспечение является автономной системой, то данный документ должен являться спецификацией».

В.3. Содержание

В данном разделе приводятся детали, проясняющие утверждение, согласно которому SRS, выполненная в соответствии с данными рекомендациями, будет также «совместимой на уровне документа» с SRD, описанной в IEEE/EIA 12207.1-1997. Требования к совместимости на уровне документа подытожены в единственной строке в таблице 1 IEEE/EIA 12207.1-1997. Эта таблица воспроизведена в таблице В.1 данных рекомендаций.

Таблица В.1 — сводка требований к SRD, извлеченная из Таблицы 1 IEEE/EIA 12207.1-1997

Информационный элемент	Раздел IEEE/EIA 12207.0-1996	Тип	Раздел IEEE/EIA 12207.1-1997	Ссылки
------------------------	------------------------------	-----	------------------------------	--------

Software Requirements Description	5.1.1.4, 5.3.4.1, 5.3.4.2	Описание (см. примечание к п. 6.22.1 IEEE/EIA 12207.1-1997)	6.22	IEEE Std 830-1998; EIA/IEEE J-STD-016, F.2.3, F.2.4; MIL-STD 961D. Также см. руководство по использованию нотаций в ISO/IEC 5806, 5807, 6593, 8631, 8790 и 11411
-----------------------------------	---------------------------	---	------	--

Требования к совместимости на уровне документа обсуждаются в следующих подразделах:

- В.3.1 описывает соответствие информационным требованиям, перечисленным во второй колонке таблицы В.1.
- В.3.2 описывает соответствие общего содержания (тип документа), как приведено в третьей колонке таблицы В.1.
- В.3.3 описывает соответствие специфическим требованиям «Описания требований к программному обеспечению», как указано в четвертой колонке таблицы В.1.
- В.3.4 описывает соответствие данным жизненного цикла в соответствии с приложением Н к IEEE/EIA 12207.0-1996, как описано в подразделе 4.2 IEEE/EIA 12207.1-1997.

В.3.1. Соответствие информационным требованиям IEEE/EIA 12207.0-1996

Информационные требования к SRD изложены в п.п. 5.1.1.4, 5.3.4.1 и 5.3.4.2 IEEE/EIA 12207.0-1996. Эти требования в основном идентичны рассмотренным в пункте В.3.3 данных рекомендаций.

В.3.2. Соответствие указаниям к общему содержанию IEEE/EIA 12207.1-1997

В соответствии с IEEE/EIA 12207.1-1997 общее содержание SRD обычно представляет собой описание, как указано в п. 5.1 IEEE/EIA 12207.1-1997. Соответствующее описание должно достигать цели, установленной в п. 5.1.1, и включать информацию, перечисленную в п. 5.1.2 IEEE/EIA 12207.1-1997.

Целью описания является:

IEEE/EIA 12207.1-1997, подраздел 5.1.1: Цель: Описать планируемую или действительную функцию, разработку, производительность или процесс.

SRD, выполненное в соответствии с данными рекомендациями, будет достигать поставленной цели.

Любые описание или спецификация, выполненные в соответствии с IEEE/EIA 12207.1-1997, будут удовлетворять требованиям к общему содержанию, установленными в п. 5.1.2 данного стандарта. В таблице В.2 данных рекомендаций перечислены общие элементы содержания и ссылки,

где это применимо, на раздел данных рекомендаций, который требует ту же информацию.

Таблица В.2 — охват IEEE Std 830-1998 требований к общему описанию

IEEE/EIA 12207.1-1997, общее содержание	Соответствующие разделы IEEE Std 830-1998	Дополнения к требованиям IEEE Std 830-1998
a) Дата выпуска, статус	–	Следует указать дату выпуска и статус
b) Область применения	5.1.2. Область применения	–
c) Выпустившая организация	–	Следует указать выпустившую организацию
d) Ссылки	5.1.4. Ссылки	–
e) Контекст	5.1.1. Назначение	–
f) Нотации для определения	4.3. Характеристики хорошей SRS	–
g) Тело	5. Составные части SRS	–
h) Резюме	5.1.1. Обзор	–
i) Глоссарий	5.1.3. Определения	–
j) История изменений	–	Следует привести историю SRD либо ссылку на нее

В.3.3. Соответствие требованиям к специфическому содержанию IEEE/EIA 12207.1-1997

Требования к специфическому содержанию SRD в IEEE/EIA 12207.1-1997 приведены в п. 6.22. SRD должно соответствовать цели, указанной в п. 6.22.1 IEEE/EIA 12207.1-1997.

Целью SRD является:

IEEE/EIA 12207.1-1997, подраздел 6.22.1: Цель: Задать требования к элементу программного обеспечения и методы, которые следует использовать, чтобы убедиться, что эти требования удовлетворены. Используется как основа для разработки и квалификационного тестирования элемента программного обеспечения.

SRS, выполненная в соответствии с данными рекомендациями и удовлетворяющая дополнительным требованиям, приведенным в таблице В.3, будет соответствовать поставленной цели.

SRD, совместимое с IEEE/EIA 12207.1-1997, должно удовлетворять специфическим требованиям к содержанию, приведенных в разделах 6.22.3 и 6.22.4 данного стандарта. В таблице В.3 данных рекомендаций перечислены специфические элементы содержания и ссылается на раздел данных рекомендаций, который требует ту же информацию, если это применимо.

Если SRD специфицируется в соответствии с требованиями, которые устанавливает или на которые ссылается таблица В.3 данных рекомендаций, оно должно разрабатываться с учетом критериев, приведенных в IEEE/EIA 12207.0-1996.

Таблица В.3 — охват специфических требований к SRD IEEE Std 830-1998

Специфическое содержание IEEE/EIA 12207.1-1997	Соответствующие разделы IEEE Std 830-1998	Дополнения к требованиям IEEE 830-1998
а) Общее описание	См. таблицу В.2	—
б) Идентификация и обзор системы	5.1.2. Область применения	—
в) Функциональность элемента программного обеспечения, включая: - требования к производительности; - физические характеристики; - внешние условия.	5.3.2. Функции 5.3.3. Требования к производительности	Следует указать физические характеристики и внешние условия.
д) Требования к внешним интерфейсам элемента программного обеспечения	5.3.1. Внешние интерфейсы	—
е) Квалификационные требования	—	Следует указать требования, используемые для квалификационного тестирования (или сослаться на них).

f) Спецификации безопасности	5.2.4. Ограничения	—
g) Спецификации секретности и приватности	5.3.6.3. Безопасность	—
h) Требования к эргономике	5.2.3. Характеристики пользователя 5.2.1.2. Интерфейсы пользователя	—
i) Определение данных и требования к базе данных	5.3.4. Логические требования к базе данных	—
j) Требования к установке и приемке на рабочем объекте	5.2.1.8. Требования к адаптации на объекте	Требования к установке и приемке на рабочем объекте
k) Требования к установке и приемке на объекте поддержки	—	Требования к установке и приемке на рабочем объекте
l) Требования к пользовательской документации	—	Требования к пользовательской документации
m) Пользовательские требования времени выполнения	5.2.1.7. Операции	Пользовательские требования времени выполнения
n) Пользовательские требования к поддержке	5.3.6.4. Поддерживаемость	—
o) Характеристики качества программного обеспечения	5.3.6. Системные атрибуты программного обеспечения	—
p) Ограничения проектирования и реализации	5.2.4. Ограничения	—
q) Требования к ресурсам компьютера	5.3.3. Требования к производительности	Требования к ресурсам компьютера
r) Требования к упаковке	—	Требования к упаковке
s) Последовательность и критичность требований	5.2.6. Распределение требований	—

t) Трассируемость требований	4.3.8. Трассируемость	—
u) Логические обоснования	5.2.5. Предположения и зависимости	—
Элементы с а) по f), перечисленные ниже, взяты из 6.22.4.		
a) Поддержка данных жизненного цикла согласно Приложения Н к IEEE/EIA 12207.0-1996	—	Поддержка данных жизненного цикла согласно Приложения Н к IEEE/EIA 12207.0-1996
b) Описание каждой функции с использованием хорошо определенной нотации	4.3. Характеристики хорошей SRS	—
c) Отсутствие конфликтующих требований	4.3. Характеристики хорошей SRS	—
d) Стандартная пользовательская терминология и определения	5.1.3. Определения	—
e) Определять каждое требование уникально для предотвращения несогласованности	4.3. Характеристики хорошей SRS	—
f) Уникально идентифицировать каждое требование	4.3. Характеристики хорошей SRS	—